

Python mission guide: braking, stress_area & custom detection models

How to brake and stop a mission, produce scoreable stressed-crop polygons, and install your own ONNX detector for a Python autonomy mission.

What you need to know when you write the mission in Python (`local_planner` / `skytrack_autonomy`). The example mission, the sample model and this guide are in `contests/skytrack-hackathon-2026/`.

1. Braking, pausing and stopping a mission
2. Writing `stress_area` from a Python mission
3. Using your own detection model

Braking, pausing and stopping a mission

There is no separate "break" call. A mission is a Python generator, so you brake with a step, leave it with plain Python, and an operator stops it with a ROS topic.

Brake: stop and hover

`brake()` is a step like `fly_to()`. It holds the drone where it is until its horizontal speed has settled, then the mission goes on to the next step.

PYTHON

```
from local_planner import brake, fly_to, land, takeoff

def my_mission(ctx):
    yield takeoff(alt_m=5.0)
    yield fly_to(north=50.0, east=0.0, alt_m=5.0)
    yield brake() # hover until the drone has settled
    yield land()
```

Always `yield brake()` right before `land()`, and before a `capture()` if you want a sharp frame.

End the mission early, from inside it

Use Python: `break` leaves a loop, `return` ends the mission. Land before you leave. A `try` / `finally` makes the landing happen however the loop ends:

PYTHON

```
def my_mission(ctx):
    yield takeoff(alt_m=5.0)
    try:
        for x, y in WAYPOINTS_ENU:
            if ctx.senses.battery.remaining < 0.3:
                break # stop early
            yield fly_to(north=y, east=x, alt_m=5.0)
    finally:
        yield brake()
        yield land()
```

Pause, cancel or stop it from outside

The mission node listens on three topics. Each one overrides whatever the mission is doing.

Topic (<code>std_msgs/Bool</code>)	<code>data: true</code>	<code>data: false</code>
<code>/pause_mission</code>	Holds the drone where it is	The mission carries on
<code>/cancel_mission</code>	Cancels the mission	Clears the cancel, ready for a new mission
<code>/emergency_stop</code>	Hard stop, then the drone lands where it is	Releases the stop

From a terminal on your machine, with the simulation running:

SHELL

```
C=skytrack-simulation-skytrack-autonomy-1
docker exec $C bash -lc 'source /opt/ros/jazzy/setup.bash && \
ros2 topic pub --once /pause_mission std_msgs/msg/Bool "{data: true}"'
```

`/emergency_stop` is for emergencies: it lands wherever the drone happens to be.

Writing `stress_area` from a Python mission

When the mission is written in Python, **the framework does not produce the stressed-crop areas for you**. You have to write the detection result to one specific file. After the mission ends the desktop app collects that file and attaches it to the mission report at `status_summary.extras.stress_area`. If the file is missing, in the wrong place or in the wrong format, the report carries no `stress_area` and the detection part scores nothing.

A complete example ships next to this guide: [hackathon_example_v1.py](#).

Why Python code mode

The AI model in the UI only supports **detection models** (bounding boxes), not **segmentation**. A `stress_area` is defined by the outline (polygon) of the stressed patch, so if you want the real shape instead of a rectangle, use **Python code mode**: run your own model or algorithm, work out the stressed areas in ENU coordinates, and write the polygons yourself.

The path

```
/root/.ros/captures/stress_area.json
```

This is the only path that gets collected. A file written anywhere else is not attached.

Notes:

- Create the parent directory yourself before writing (`parents=True, exist_ok=True`).
- There must be exactly **one** file named `stress_area.json` under `/root/.ros/captures`, subdirectories included. Do not leave backups or drafts with that name anywhere in there.
- Write **atomically**: write to a temp file under a different name (for example `.stress_area.json.tmp`), then `replace` it onto the real name. That way a mission stopped mid-write leaves the previous, still valid file instead of a truncated one.
- You may write several times during a mission, for instance after each area. Whatever is on disk when the mission ends is what gets collected.

- **Write before landing** — right after the survey stage, for example. Never after `land()`.

Format

JSON

```
{
  "stress_area": [
    {
      "class": "stressed",
      "polygon": [[x1, y1], [x2, y2], [x3, y3]]
    }
  ]
}
```

Field	Type	Meaning
<code>stress_area</code>	list	The areas you found; <code>[]</code> when there are none
<code>class</code>	string	Area label, use <code>"stressed"</code> . Missing means <code>"stressed"</code>
<code>polygon</code>	list of <code>[x, y]</code>	The outline, in world ENU metres (<code>x</code> east, <code>y</code> north), at least 3 vertices

Also:

- The top level **must** be the object `{"stress_area": [...]}`. Do not write a bare list, and do not add other keys.
- The file must be valid JSON, UTF-8 encoded.
- A polygon does **not** need to repeat its first vertex at the end; the scorer closes the ring.
- Coordinates are world ENU, not NED and not image pixels. The pose is NED, so convert: `east, north = pose.y, pose.x`.
- Polygons with fewer than 3 vertices are dropped. Filter out tiny areas yourself to avoid noise.

Where this happens in the example

File: `hackathon_example_v1.py`

- 1 The path, declared once: [line 38](#)

PYTHON

```
STRESS_AREA_PATH = Path("/root/.ros/captures/stress_area.json") # attached to the report by the app
```

- 2 The writer — atomic, and the shape of the file: [lines 110–119](#)

PYTHON

```
def save_stress_areas(areas: list[dict[str, Any]]) -> None:
    STRESS_AREA_PATH.parent.mkdir(parents=True, exist_ok=True)
    tmp = STRESS_AREA_PATH.with_name(".stress_area.json.tmp")
    tmp.write_text(json.dumps({"stress_area": areas}, separators=(",", ":")))
    tmp.replace(STRESS_AREA_PATH)
```

- 3 Called after the survey, **before landing**: [lines 82–83](#)

PYTHON

```
areas = stress_map.stress_areas()
save_stress_areas(areas)
```

- 4 Where each entry comes from: [lines 177–189](#). `StressMap.stress_areas()` returns `[{"class": "stressed", "polygon": [[x, y], ...]}, ...]` with `x, y` already converted to ENU metres.

Replace `StressMap` with your own detection, but keep the path, the format and the way the file is written.

Using your own detection model

The UI's AI model list is fixed. To detect with a model of your own — or with the sample one — put its two files, the ONNX weights and a catalogue JSON, in the drone's user model folder, then ask for it by name from your Python mission.

The sample model

A ready-made sample ships next to this guide: [sample_ai_model.zip](#).

File in the zip	What it is
<code>sample_ai_model/det-h2026-v26n-b-fp32-640.onnx</code>	Detector for stressed crop, 640×640, fp32
<code>sample_ai_model/sample.json</code>	Its catalogue: one dataset class, <code>stressed</code>

Bring your own model in the same shape:

- **ONNX, end-to-end export.** Output `[1, N, 6] = (x1, y1, x2, y2, score, class_id)` in input pixels. The detector runs no NMS of its own.
- **The weights are named after the model id:** `<model id>.onnx`, where the id is the key under `models` in the JSON. Ids may only use letters, digits, `.`, `_` and `-`, and must not be the id of a built-in model.
- **The JSON declares the model's dataset** in its own `dataset` section, with a non-empty `classes` list. Do not reuse the name of a built-in dataset with different classes.
- **classes order is class_id order:** `classes[0]` is class id 0, and so on.
- **Each object_mapping entry is an object** with a `models` list, as in `sample.json` — not a bare list.
- **Ask for a class by its exact name.** The `classes` you request must be names from the dataset's `classes` (case does not matter). `object_mapping` is only a hint for the UI.

1. Copy the two files into the user model folder

The folder is `/tmp/skytrack-session/data` in the drone container. The detector reads every `*.json` in it as a catalogue, and takes a model's weights from `<model id>.onnx` in the same folder. From a terminal on your machine, with the simulation running:

SHELL

```
unzip sample_ai_model.zip
C=skytrack-simulation-skytrack-autonomy-1
docker exec $C mkdir -p /tmp/skytrack-session/data
docker cp sample_ai_model/det-h2026-v26n-b-fp32-640.onnx $C:/tmp/skytrack-session/data/
docker cp sample_ai_model/sample.json $C:/tmp/skytrack-session/data/
```

- Put both files straight in the folder, side by side, not in a subfolder.
- Nothing needs restarting: the detector picks the model up on its next request. Copying new weights under the same name takes effect on the next request too.
- The built-in models stay available.

- `/tmp` is reset whenever the app recreates the containers. Check the files are still there before each run, and copy them again if not:

SHELL

```
docker exec $C ls /tmp/skytrack-session/data
```

- A file that is not accepted — broken JSON, an id already taken, a dataset missing — is skipped with a warning in the detector's log, and asking for that model then fails with `Unknown model`.

2. Detect with it

Register the `Detector` service with your model before the drone flies:

PYTHON

```
from local_planner import boot_drone
from skytrack_autonomy import Detector

MODEL_NAME = "det-h2026-v26n-b-fp32-640"
CLASSES = ["stressed"]

def main() -> None:
    with boot_drone() as drone:
        drone.add_service(Detector(model_name=MODEL_NAME, classes=CLASSES))
        drone.fly(my_mission)
        drone.run()
```

Inside the mission, request a detection and read it back once it has answered:

PYTHON

```
detector = ctx.services.detector
before = detector.count
detector.request(model_name=MODEL_NAME, classes=CLASSES, confidence_threshold=0.4)
# ... hover until detector.count > before (the detector never moves the drone) ...
outcome = detector.last_result
for det in outcome.detections:
    print(det.class_name, det.score, det.bbox.center_x, det.bbox.center_y,
          det.bbox.size_x, det.bbox.size_y)
```

Every box comes back with `class_name == "stressed"`. Boxes are in **image pixels** (`outcome.image_width` × `outcome.image_height`), not world coordinates: turning them into `stress_area` polygons is up to you — project them through the nadir camera the same way `StressMap.add_picture` does in [hackathon_example_v1.py](#).